
CENTRO NACIONAL DE INVESTIGACIÓN, DOCUMENTACIÓN
E INFORMACIÓN MUSICAL “CARLOS CHÁVEZ”

Cancionero Infantil CENIDIM

Manual del sistema

*Plataforma web de consulta, análisis y administración
del archivo de letras de canciones*

Versión del documento: 1.0
Versión del sistema: 0.2.0
Fecha: Septiembre de 2026
Rama de referencia: release
Despliegue: `cenidim.darylemb.dev`

Documento entregable del proyecto.

Describe la arquitectura, la instalación, el uso, la referencia de la API, el estado de calidad verificado y el trabajo pendiente.

Índice general

1. Introducción	1
1.1. Propósito del sistema	1
1.2. Cifras del corpus	1
1.3. Alcance de este manual	1
2. Arquitectura del sistema	2
2.1. Vista general	2
2.2. Componentes	2
2.2.1. Backend — FastAPI / Pydantic v2	2
2.2.2. Frontend — Vue 3 + TypeScript	2
2.2.3. Datos	3
2.3. Decisiones de arquitectura registradas	3
3. Pipeline de datos	4
4. Instalación y despliegue	5
4.1. Requisitos	5
4.2. Despliegue con Docker (recomendado)	5
4.3. Desarrollo local sin Docker	5
4.4. Variables de entorno	5
4.5. Autenticación	6
5. Manual de usuario	7
5.1. Vistas de la aplicación	7
5.1.1. Línea de tiempo	7
5.1.2. Catálogo (<i>Canciones</i>)	7
5.1.3. Tablero estadístico	7
5.1.4. Panel de administración	7
6. Referencia de la API	8
6.1. Públicos	8
6.2. Autenticación (/api/auth/*)	8
6.3. Administración (/api/admin/*)	8
7. Calidad, pruebas e integración continua	9
7.1. Estado verificado	9
7.2. Puertas de calidad configuradas	9
7.3. Jobs de CI (GitHub Actions)	9
8. Estado del proyecto y trabajo pendiente	10
8.1. Veredicto	10
8.2. Resueltos en la rama release	10
8.3. Pendientes técnicos	10
8.4. Pendientes editoriales (con el equipo del archivo)	11

8.5. Riesgos y notas de operación	11
A. Glosario breve	12
Referencias	13

Índice de tablas

1.1. Volumen del archivo al momento de esta edición.	1
4.1. Variables de entorno del backend.	6
5.1. Mapa de rutas del frontend.	7
6.1. Endpoints públicos.	8
6.2. Endpoints de autenticación.	8
7.1. Panel de calidad. *Vitest 4; el ajuste de <code>localStorage</code> de <code>src/test/setup.ts</code> ya forma parte del árbol, sin workarounds de entorno.	9
8.1. Pendientes técnicos restantes tras la rama release	11

Introducción

1.1. Propósito del sistema

La plataforma **Cancionero CENIDIM** es una aplicación web que da acceso público y administrado al archivo digital del cancionero infantil del Centro Nacional de Investigación, Documentación e Información Musical “Carlos Chávez”. Permite:

- Buscar canciones por título, álbum (fonograma) o contenido de la letra.
- Explorar el catálogo en una *línea de tiempo* por año de registro.
- Consultar un *tablero estadístico* con métricas del corpus: distribución por año, por tema, por clasificación lingüística, niveles de léxico (OOV) y nube de palabras.
- Administrar el catálogo (fonogramas y canciones), las cuentas de usuario y la auditoría operativa (roles *viewer/editor/admin*).

1.2. Cifras del corpus

Indicador	Valor
Canciones indexadas	3 858
Fonogramas (álbumes)	280
Temas catalogados (distintos)	~24 canónicos
Clasificaciones lingüísticas	3 (estándar, regional, indígena)

Tabla 1.1: Volumen del archivo al momento de esta edición.

Los temas (**Tema**: en los archivos fuente) son asignados por catalogadores humanos; el sistema *no* los infiere. Las variantes de escritura (mayúsculas, espacios, errores tipográficos conocidos) se pliegan a formas canónicas mediante `canonical_tema` (`backend/app/models/theme_normalization.py`).

1.3. Alcance de este manual

Este documento es el manual entregable del proyecto. Cubre la arquitectura (cap. 2), el pipeline de datos (cap. 3), la instalación y el despliegue (cap. 4), el uso de la aplicación (cap. 5), la referencia de la API (cap. 6), el estado de calidad verificado (cap. 7) y el trabajo pendiente (cap. 8).

Arquitectura del sistema

2.1. Vista general

El sistema sigue una arquitectura de tres capas desplegada con Docker Compose:

Servicio	Puerto	Función
frontend	80	SPA Vue 3 servida por Nginx (sin root)
backend	8000	API FastAPI / uvicorn
db-init	—	Sidecar que regenera <code>letras.db</code>

El frontend consume la API bajo el prefijo `/api` (proxy Nginx en producción, proxy Vite en desarrollo). La base de datos es **SQLite** (`letras.db`), regenerada de forma determinista por el sidecar `db-init` a partir de las fuentes, por lo que el archivo binario es un artefacto de ejecución, no una fuente editada a mano.

2.2. Componentes

2.2.1. Backend — FastAPI / Pydantic v2

- SQLAlchemy 2.0 (ORM) sobre el esquema `letras.db` (columnas `snake_case`); modelos en `app/models/`.
- Autenticación: hash de contraseñas `sha256+bcrypt`, JWT con `python-jose`, cookies `HttpOnly` + CSRF *double-submit*, rotación de refresh tokens con revocación (tabla `RefreshTokenRevocation`).
- Autorización por roles: `viewer < editor < admin`, vía la fábrica `require_role(...)` en `app/deps.py`.
- Observabilidad: `/metrics` en formato Prometheus y logging JSON estructurado (`app/observability.py`, `app/logging_config.py`).
- Migraciones con Alembic; en producción (`CENIDIM_ENV=prod`) el esquema *no* se autocrea.
- Correo (recuperación de contraseña) vía Resend; en desarrollo, bandeja local en la tabla `email_outbox`.

2.2.2. Frontend — Vue 3 + TypeScript

- SPA con Vite, Vue Router 4, Pinia (stores `auth`, `search`, `ui`, `filters`).
- Gráficas con `vue-chartjs` (Chart.js).
- TypeScript en modo *strict*; estilos con design tokens (`src/assets/tokens.css`: Fraunces + Outfit + JetBrains Mono sobre paleta vino/mostaza/crema).

2.2.3. Datos

- Fuentes: `db_fonografia.csv` (metadata de fonogramas) y `LetrasTXT/*.txt` (letras con línea Tema:).
- Artefacto: `letras.db` (SQLite) montado en `/data/letras.db` dentro del contenedor.

2.3. Decisiones de arquitectura registradas

El backend original en Go/Gin fue reemplazado por FastAPI y retirado en la “Fase 9” del plan de migración. La decisión y sus consecuencias se documentaron en el ADR `docs/adr/0001-fastapi-replaces-go.md` y el runbook `docs/CUTOVER.md`; ambos archivos fueron retirados del árbol en la limpieza posterior (commit `ff4303b`) y permanecen accesibles en la historia de git. El backend de Go permanece disponible en la historia de git (commit `2aab765`) solo como referencia de emergencia.

Pipeline de datos

La base de datos se construye en tres pasos encadenados por `scripts/build_db.sh` (o por el contenedor `db-init`):

1. **Construcción** — `scripts/build_db.py`: parsea `db_fonografia.csv` + `LetrasTXT/` y genera `letras.db` (port a Python del antiguo builder en Go, con salida byte-compatible). Crea el usuario administrador inicial si se define `ADMIN_PASS`.
2. **Clasificación** — `scripts/classify_songs.py`: con spaCy (`es_core_news_md`) calcula el porcentaje de palabras fuera de vocabulario (OOV) de cada letra y escribe `song_stats`:

Clasificación	Criterio
ESPAÑOL_ESTANDAR	OOV < 5 %
ESPAÑOL_REGIONAL	$5 \% \leq \text{OOV} \leq 18 \%$
LENGUA_INDIGENA	OOV > 18 % o léxico indígena

3. **Normalización** — `scripts/normalize_db.py`: limpia el cuerpo de las letras (cabeceras, líneas `Dura:/Tema:/Personajes:`, iniciales), normaliza temas y revalida la correspondencia letra $\leftrightarrow$ título. Es idempotente y conservadora (verificada con tests).

Puntos de integridad verificados. Tras la normalización, 366 letras fueron limpiadas y 0 conservan marcadores de metadatos residuales; el título y autor permanecen en sus columnas dedicadas, por lo que la nube de palabras y el léxico OOV reflejan únicamente el texto de la canción.

Instalación y despliegue

4.1. Requisitos

- Docker + Docker Compose (vía recomendada), o
- Python $\geq 3,12$ con uv; Node.js 24; spaCy `es_core_news_md` para regenerar la base de datos.

4.2. Despliegue con Docker (recomendado)

```
docker compose up --build -d
backend/scripts/smoke.sh http://localhost:8000
```

Servicios resultantes:

- Frontend: `http://localhost` (Nginx no privilegiado).
- API: `http://localhost:8000/healthz`; métricas en `/metrics`; especificación en `/openapi.json`.

Para producción (Coolify) se usa `docker-compose-coolify.yaml` con volumen nombrado y secretos del gestor; el archivo por defecto sigue siendo válido.

4.3. Desarrollo local sin Docker

```
# Backend
cd backend
uv sync
PYTHONPATH=. uv run pytest tests/
uv run uvicorn app.main:app --port 8000 --reload

# Frontend (Node 24)
cd frontend
npm install
npm run dev          # Vite en :5173, proxy /api -> :8000
```

4.4. Variables de entorno

Variable	Requerida	Descripción
CENIDIM_JWT_SECRET	Sí (prod)	Secreto de firma JWT (≥ 256 bits; <code>openssl rand -hex 32</code>). Sin él el backend no arranca en producción.

Variable	Requerida	Descripción
CENIDIM_DB_PATH	No	Ruta a <code>letras.db</code> (def. <code>letras.db</code>).
CENIDIM_ENV	No	<code>dev</code> permite autocrear el esquema; <code>prod</code> exige Alembic.
CENIDIM_LOG_FORMAT	No	<code>json</code> (producción) o <code>text</code> .
CENIDIM_WORKERS	No	Workers de <code>uvicorn</code> (def. 2).
CORS_ALLOWED_ORIGINS	No	Orígenes permitidos, separados por comas.
RESEND_API_KEY	Para correo	Sin ella, los envíos caen en la tabla <code>email_outbox</code> .
CENIDIM_EMAIL_FROM	Para correo	Remitente de correos de recuperación.
FRONTEND_BASE_URL	Para correo	Origen del SPA para los enlaces de restablecimiento.
ADMIN_PASS	Al sembrar	Contraseña del administrador inicial (<code>build_db</code>).

Tabla 4.1: Variables de entorno del backend.

4.5. Autenticación

El inicio de sesión es usuario + contraseña (Google OAuth fue retirado en la migración a FastAPI). `POST /api/auth/login` emite las cookies `HttpOnly` `cenidim_session` (JWT de acceso), `cenidim_refresh` y `cenidim_csrf`. Los endpoints `/forgot` y `/reset` implementan recuperación por correo (Resend o bandeja de desarrollo).

Manual de usuario

5.1. Vistas de la aplicación

Ruta	Vista	Acceso
/	Línea de tiempo	Público
/canciones	Catálogo buscable	Público
/dashboards	Tablero estadístico	Público
/admin	Panel de administración	JWT + rol
/login	Inicio de sesión	Público
/forgot, /reset	Recuperación	Público

Tabla 5.1: Mapa de rutas del frontend.

5.1.1. Línea de tiempo

Agrupar las canciones por año de registro del fonograma y permite abrir cada pieza para leer su letra y metadatos.

5.1.2. Catálogo (*Canciones*)

Búsqueda paginada por título, álbum o contenido; filtros combinables por tema, rango de años, clasificación lingüística y fonograma. La columna de número no se trunca y el ordenamiento no produce parpadeo.

5.1.3. Tablero estadístico

KPIs del catálogo (canciones indexadas, álbumes en el filtro activo, temas distintos), gráficas por año, por tema, por clasificación y por nivel OOV, y nube de palabras por frecuencia. Cada gráfica tiene un ícono ⓘ con explicación en lenguaje no técnico; el texto fuente canónico es `frontend/src/config/chartInfo.ts` (el glosario editorial `docs/GLOSARIO.md` fue retirado del árbol en la limpieza de `ff4303b`; disponible en la historia de git).

5.1.4. Panel de administración

CRUD de fonogramas, canciones y usuarios con paginación y ordenamiento del lado del servidor; consulta de la bandeja de correo (`email_outbox`) y de la bitácora de auditoría. La jerarquía de roles es `viewer` (lectura) < `editor` (escritura) < `admin` (borrado y gestión de usuarios).

Referencia de la API

La especificación completa y normativa es `backend/openapi.json` (OpenAPI 3.1, regenerable con `scripts/generate_openapi.py` y vigilada por un guardia de deriva en CI). Resumen:

6.1. Públicos

Endpoint	Descripción
GET <code>/healthz</code>	Health check (Docker).
GET <code>/metrics</code>	Métricas Prometheus.
GET <code>/openapi.json</code>	Especificación OpenAPI 3.1.
GET <code>/api/search</code>	Búsqueda paginada; <code>?query=</code> o <code>?q=</code> .
GET <code>/api/song/{id}</code>	Detalle y letra de una canción.
GET <code>/api/timeline</code>	Canciones agrupadas por año.
GET <code>/api/stats</code>	Métricas del tablero; honra los filtros compartidos (<code>tema</code> , <code>year_from</code> , <code>year_to</code> , <code>clasificacion</code> , <code>album</code> , <code>q</code>).
GET <code>/api/word-cloud</code>	Frecuencias de palabras (def. 200, máx. 500 vía <code>?limit=</code>).

Tabla 6.1: Endpoints públicos.

6.2. Autenticación (`/api/auth/*`)

Endpoint	Descripción
POST <code>/login</code>	Emite cookies de sesión + <code>{token, user}</code> .
POST <code>/register</code>	Auto-registro (si está habilitado).
POST <code>/forgot</code>	Siempre 200; envía enlace de un solo uso.
POST <code>/reset</code>	Consume el token y fija la nueva contraseña.
POST <code>/refresh</code>	Rota el refresh token (revoca el <code>jti</code> anterior).
POST <code>/logout</code>	Revoca el refresh token y limpia cookies.
GET <code>/me</code>	Perfil del usuario autenticado.

Tabla 6.2: Endpoints de autenticación.

6.3. Administración (`/api/admin/*`)

Requieren JWT y rol suficiente. CRUD completo de `fonogramas`, `songs` y `users`; más los operativos GET `/emails` (bandeja, filtro `only_failures`) y GET `/audit` (bitácora, filtros `actor_id`, `action`).

Calidad, pruebas e integración continua

7.1. Estado verificado

Los siguientes resultados fueron *ejecutados y verificados* sobre la rama **release** en la fecha de este documento:

Verificación	Comando	Resultado
Tests backend	<code>pytest tests/</code>	✓ 235 pasan, cob. 83.38 %
Lint backend	<code>ruff check app/ tests/</code>	✓ limpio
Tipos frontend	<code>vue-tsc -noEmit</code>	✓ limpio
Lint frontend	<code>npm run lint</code>	✓ 0 errores, 17 advertencias
Tests frontend	<code>vitest -run</code>	✓ 265 pasan, 9 omitidos*
Tipos backend	<code>mypy app/</code>	✓ limpio (33 archivos)

Tabla 7.1: Panel de calidad. *Vitest 4; el ajuste de `localStorage` de `src/test/setup.ts` ya forma parte del árbol, sin workarounds de entorno.

7.2. Puertas de calidad configuradas

- Cobertura backend $\geq 80\%$ (`-cov-fail-under`, `pyproject.toml`); cobertura actual 83.38 % en corrida limpia (un comentario del propio archivo menciona un objetivo de 95 % de una fase previa del plan maestro).
- Umbrales de cobertura frontend en `vitest.config.ts` (líneas/sentencias 95 %, ramas 83 %, funciones 70 %).
- Guardia de deriva OpenAPI: `openapi.json` se regenera y se compara en CI.
- Auditoría de design tokens: `scripts/audit_design_tokens.sh` limita los valores en duro fuera de `tokens.css`.
- Escaneo de vulnerabilidades con Trivy (imágenes y filesystem) en el job de Docker.
- Humo post-despliegue: `backend/scripts/smoke.sh` (código de salida no cero al primer fallo) y verificador extremo a extremo `scripts/verify_api.py` que cruza cada endpoint contra la base en disco.
- Pre-commit: `ruff` (lint+formato), `mypy`, ganchos estándar.
- Réplica local del CI: `scripts/run_ci_local.sh`.

7.3. Jobs de CI (GitHub Actions)

`lint`, `typecheck` (build con `vue-tsc`), `test`, `fastapi-checks` (`ruff` + `pytest` + deriva OpenAPI) y `docker-build` / `docker-build-fastapi` (build + health + smoke + Trivy + verificador E2E).

Estado del proyecto y trabajo pendiente

8.1. Veredicto

El sistema es **funcionalmente completo**, está **desplegado** en producción (`cenidim.darylemb.dev`) y las suites de backend y frontend están íntegramente en verde. Los bloqueantes técnicos detectados en la revisión fueron resueltos en la rama **release** (§8.2); lo que queda es trabajo menor y las decisiones editoriales del catálogo.

8.2. Resueltos en la rama **release**

- **Tipado backend.** `mypy app/` reportaba 41 errores en 7 archivos. Fueron corregidos con anotaciones reales (no supresiones): generadores asíncronos tipados (`session_scope`, `get_db`), helpers SQL de `public.py` con `TypeVar` acotado a `Select`, `cast` a `CursorResult` para `lastrowid/rowcount`, y firma anotada del middleware CSRF. Resultado: ✓ limpio en 33 archivos, y el hook de pre-commit ya no bloquea.
- **Tests frontend en Node ≥ 24 .** El `localStorage` nativo de Node sombreaba al de `jsdom` (63 fallos). `src/test/setup.ts` instala ahora un polyfill `Storage` en memoria; los 265 tests pasan sin `NODE_OPTIONS` ni workarounds de entorno.
- **Dependencias frontend.** `npm audit` reportaba 15 vulnerabilidades (1 crítica). Se eliminó `vue-wordcloud` (sin uso: la nube es el componente propio `WordCloud.vue`), se actualizó `vitest` a la 4.1.11 (corrigiendo el mock `reset` en `auth.spec.ts` conforme al nuevo comportamiento de `restoreAllMocks`) y se aplicó `npm audit fix`. Resultado: ✓ 0 vulnerabilidades.
- **Higiene del repositorio.** Los CSV de clasificación manual se conservaron en `historical/` con un README de procedencia (no son entrada del pipeline). `frontend/dist/` dejó de trackearse (lo regenera el Dockerfile desde fuente). Las dos `letras.db` se mantienen trackeadas de forma deliberada, como referencia para nuevos mantenedores. `.gitignore` corregido.
- **CI en release.** El workflow de GitHub Actions ahora dispara también en la rama **release** (push y PR).

8.3. Pendientes técnicos

Área	Pendiente
□ Licencia	Sigue sin existir un archivo <code>LICENSE</code> ; para la entrega formal conviene elegir una (o confirmar “todos los derechos reservados” con el cliente).
□ Observabilidad	Deprecación FastAPI: <code>on_event</code> en <code>app/main.py</code> debe migrarse a manejadores <i>lifespan</i> (348 avisos en la corrida de <code>pytest</code>).

	Área	Pendiente
□	Configuración	El comentario de cobertura en <code>pyproject.toml</code> cita un objetivo de 95 % de una fase previa, pero la puerta vigente es 80 % (cobertura actual 83.38 %); conviene alinearlos.
□	Herramientas	Deriva de formato: el árbol fue formateado con el <code>ruff 0.3.x</code> que pinea <code>.pre-commit-config.yaml</code> , pero el <code>ruff</code> resuelto por <code>uv.lock (0.15)</code> reformatearía 51 archivos, por lo que <code>ruff format</code> no es puerta de CI. Conviene alinear versiones o reformatear en un commit puramente mecánico.

Tabla 8.1: Pendientes técnicos restantes tras la rama `release`.

8.4. Pendientes editoriales (con el equipo del archivo)

- Catálogo canónico de temas: colapsar conceptos distintos con nombres parecidos (los meros errores tipográficos ya se pliegan con `TEMA_TYPO_MAP`).
- Homogeneizar la terminología del tablero con los documentos de trabajo y el ensayo (las etiquetas *in-app* ya tienen explicaciones $\otimes$ (i)).
- Revisar el listado de canciones sin catalogar (documento `CANCIONES-SIN-CATALOGAR-2026-08-03.md` en `docs/`, retirado del árbol en `ff4303b` y recuperable de la historia de git) para el acervo aún sin tema.

8.5. Riesgos y notas de operación

- `CENIDIM_JWT_SECRET` es obligatorio en producción; el backend no arranca sin él.
- `letras.db` es regenerable: ante corrupción basta reiniciar el stack (`db-init` lo reconstruye).
- El rollback a Go ya no existe como árbol; solo queda el commit congelado `2aab765` en la historia.

Glosario breve

Fonograma

Registro de álbum/disco fuente del que provienen las canciones; unidad de colección (280 en el corpus).

OOV

Out Of Vocabulary: porcentaje de palabras de una letra que el modelo spaCy no reconoce. Proxy de qué tan cotidiano o especializado es el lenguaje.

Tema

Descriptor editorial asignado por catalogadores humanos (línea **Tema:** del archivo fuente). Nunca se infiere por palabras clave.

Clasificación lingüística

Tipología automática por letra: español estándar, español regional o lengua indígena (umbrales OOV en §3).

db-init

Servicio sidecar de Docker que reconstruye `letras.db` (build → clasifica → normaliza) antes de que arranque el backend.

email_outbox

Tabla-bandeja donde caen los correos de recuperación cuando no hay `RESEND_API_KEY` (desarrollo).

ADR

Architecture Decision Record; el de este proyecto es `docs/adr/0001-fastapi-replaces-go.md` (retirado del árbol en `ff4303b`; en la historia de git).

El glosario canónico completo, con la definición de cada KPI y gráfica del tablero, era `docs/GLOSARIO.md` (retirado en `ff4303b`); en la aplicación, la fuente viva es `frontend/src/config/chartInfo.ts`.

Bibliografía

- [1] Repositorio del proyecto, *README.md* (raíz) y *backend/README.md*.
- [2] ADR 0001: *FastAPI reemplaza a Go*, docs/adr/0001-fastapi-replaces-go.md (retirado del árbol en ff4303b; recuperable de la historia de git).
- [3] *Cutover Playbook — Fases 7 y 9*, docs/CUTOVER.md (ídem).
- [4] *Glosario del tablero*, docs/GLOSARIO.md (ídem; la fuente viva en la aplicación es frontend/src/config/chartInfo.ts).
- [5] *Respuestas a las 16 observaciones del tablero*, docs/REVIEW-2026-07-01-STATUS.md y docs/RESPUESTAS-CLIENTE-CENIDIM-2026-07-28.md (ídem).
- [6] Especificación OpenAPI 3.1 de la API, backend/openapi.json.
- [7] Documentación de FastAPI, <https://fastapi.tiangolo.com/>.
- [8] Documentación de Vue 3, <https://vuejs.org/>; Pinia, <https://pinia.vuejs.org/>; Vue Router, <https://router.vuejs.org/>.
- [9] spaCy, modelo español es_core_news_md, <https://spacy.io/models/es>.
- [10] Chart.js, <https://www.chartjs.org/>; envoltorio vue-chartjs.